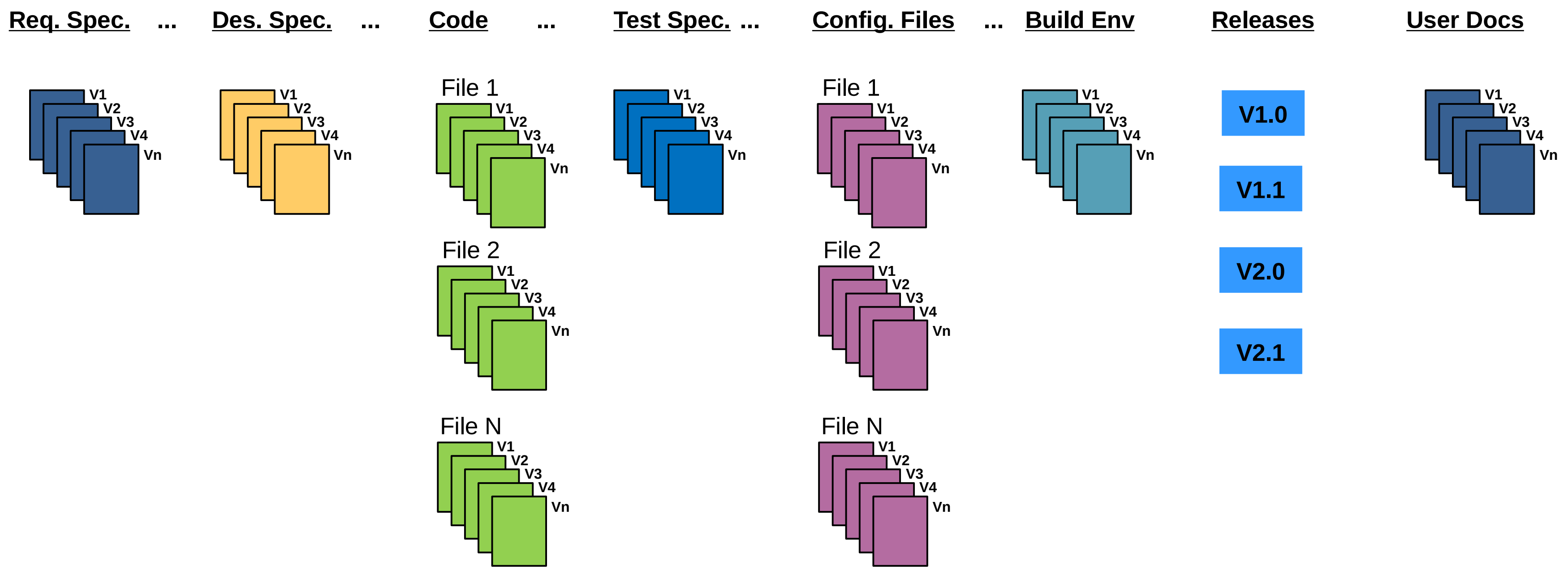
Configuration Management

Background

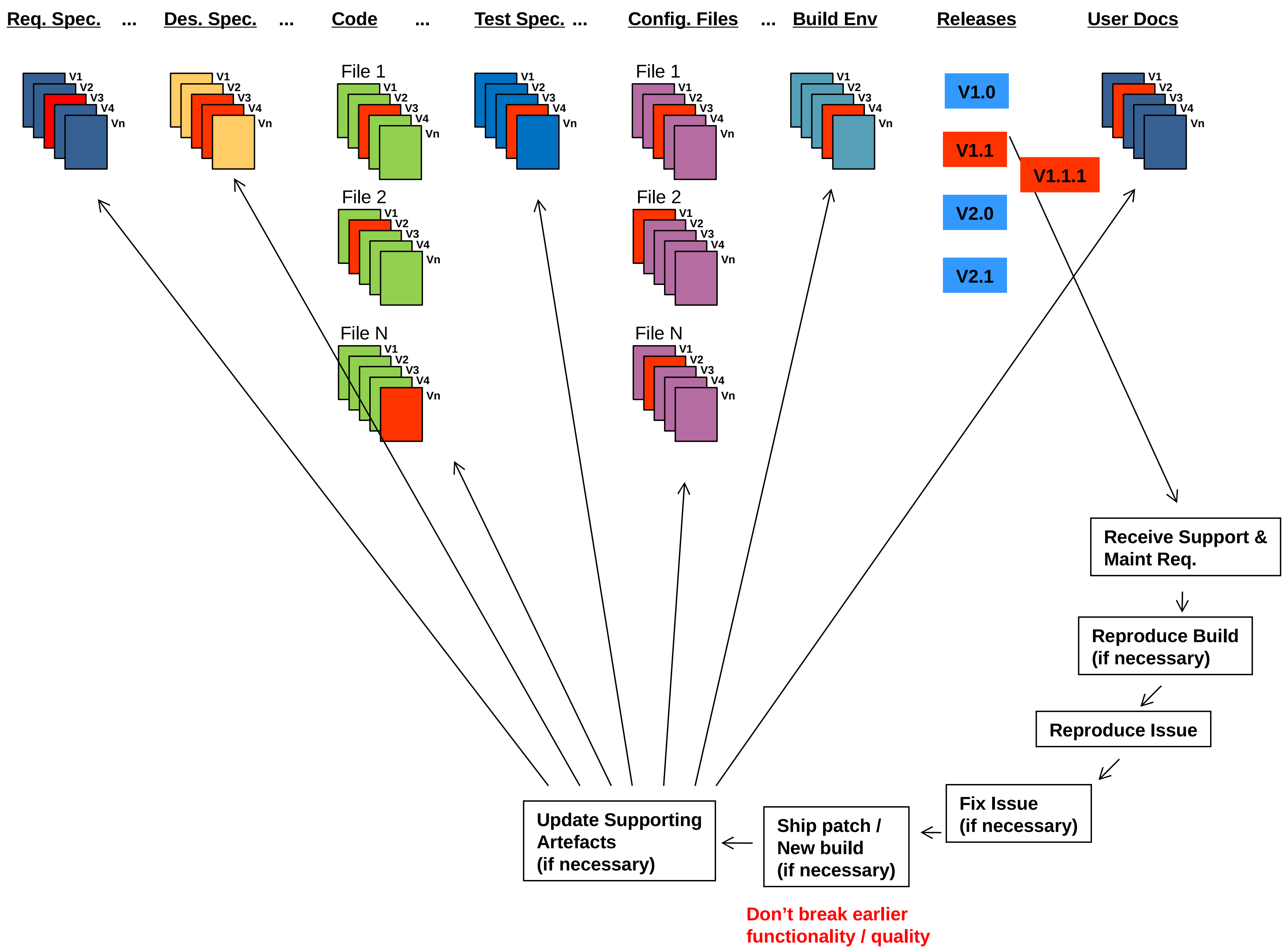
For testing, configuration management may involve ensuring that:

- All items of testware are identified, version controlled, tracked for changes, related to each other and related to development items (test objects) so that
- All identified documents and software items are referenced unambiguously in test documentation

The configuration management procedures and infrastructure (tools) should be chosen, documented and implemented.



Req. Spec. ... Des. Spec. ... Code ... Test Spec. ... Config. Files ... Build Env Releases User Docs
V1 V2 V3 V4 Vn
File 1
File 2
File N
V1.0
V1.1
V2.0
V2.1



Req. Spec. ... Des. Spec. ... Code ... Test Spec. ... Config. Files ... Build Env Releases User Docs
File 1
File 2
File N
V1 V2 V3 V4 Vn
V1.0
V1.1
V1.1.1
V2.0
V2.1
Receive Support & Maint Req.
Reproduce Build (if necessary)
Reproduce Issue
Fix Issue (if necessary)
Ship patch / New build (if necessary)
Update Supporting Artefacts (if necessary)
Don't break earlier functionality / quality

Symptoms of poor configuration management

Can't find latest version of source code or match source to object

Can't replicate previously released version of code for a customer

Bugs that were fixed suddenly reappear

Wrong functionality shipped

Wrong code tested

Undocumented features suddenly appear

Tested features suddenly disappear

Can't trace which customer has which version of code

Simultaneous changes made to same source module by multiple developers and some changes lost.

Risk and Testing

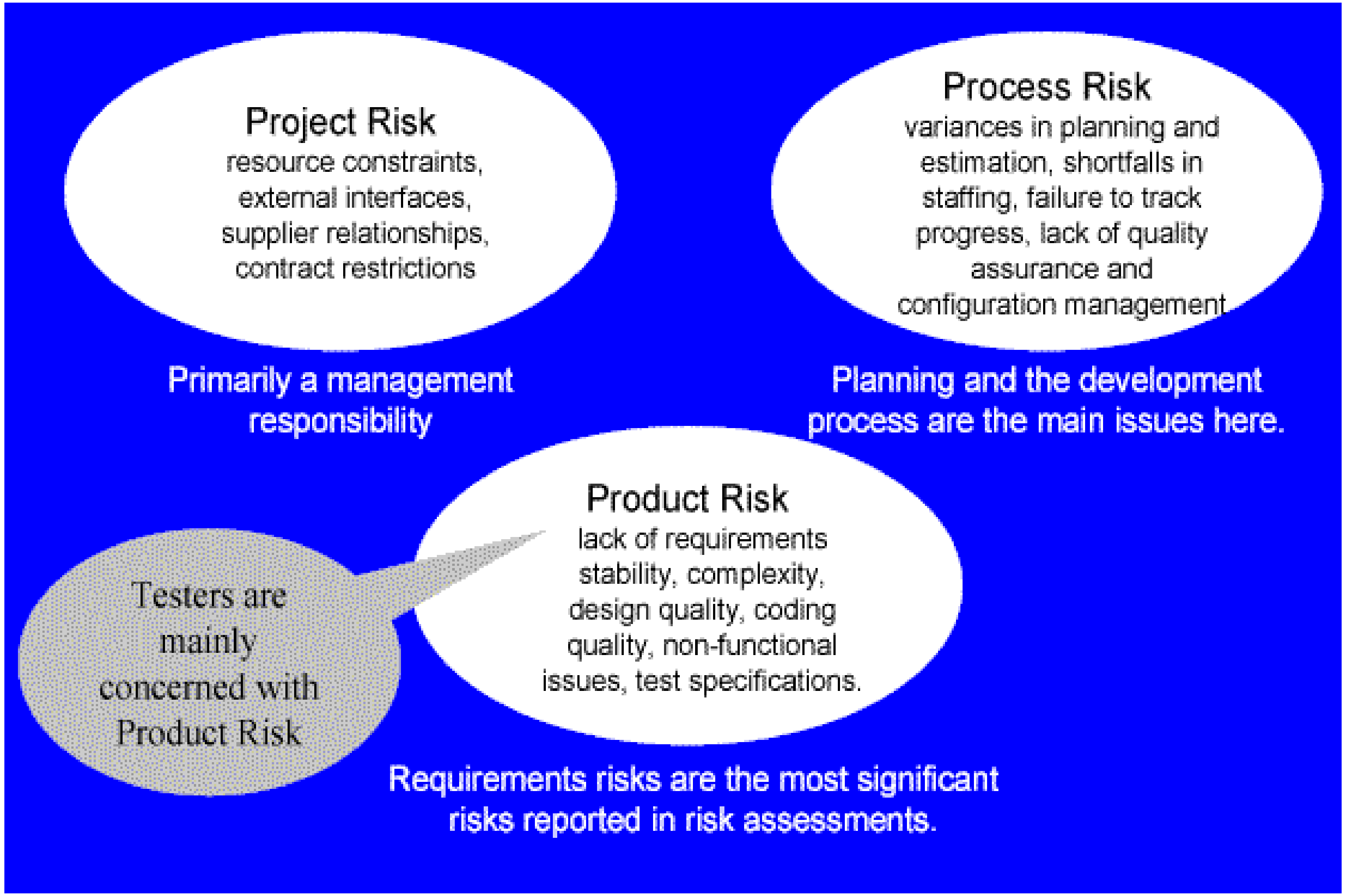
Stakeholder Objectives and Risks

- Definition: "A Stakeholder Objective is one of the fundamental objectives of the system to be built".
  - Benefits of undertaking the project
  - Payoff(s) that underpin and justify the project
- Software risks are potential events or things that threaten the Cardinal Objectives of a project.
- Definition: "A risk is a threat to one or more of the cardinal objectives of a project that has an uncertain probability".

Some general statements about risk

- Let’s explore our definition of risk
- Risks only exist where there is uncertainty
  - if the probability of a risk is zero or 100%, it is not a risk
  - unless there is the potential for loss, there is no risk
- There are risks associated with every project - software development is inherently risky.
- Risk pervades our language

Three types of software risk



Risk-Based Testing

Risks are used to decide where to start testing and where to test more; testing is used to reduce the risk of an adverse effect occurring, or to reduce its impact.

Product risks are a special type of risk to the success of a project.

Testing as a risk-control activity provides feedback about the residual risk by measuring the effectiveness of critical defect removal and of contingency plans.

Risk-Based Testing 2

- In a risk-based approach the risks identified may be used to:
  - Determine the test techniques to be employed.
  - Determine the extent of testing to be carried out.* Prioritize testing in an attempt to find the critical defects as early as possible.
  - Determine whether any non-testing activities could be employed to reduce risk (e.g. providing training to inexperienced designers).
- Risk-based testing draws on the collective knowledge and insight of the project stakeholders to determine the risks and the levels of testing required to address

Risk-Based Testing 3

To ensure that the chance of a product failure is minimized, risk management activities provide a disciplined approach to:

- Assess (and reassess on a regular basis) what can go wrong (risks).
- Determine what risks are important to deal with.
- Implement actions to deal with those risks.

In addition, testing may support the identification of new risks, may help to determine what risks should be reduced, and may lower uncertainty about risks.

Incident Management

Why track incidents?

The objectives of incident reporting:

- To provide developers and other parties with feedback about the problem to enable identification, isolation and correction as necessary.
- To provide test leaders a means of tracking the quality of the system under test and the progress of the testing.
- To provide an input to test process improvement initiatives.

When to log an incident

We log incidents when a test result appears to be different from the expected result

This could be because:

- something wrong with the test
- something wrong with the expected result
- something wrong with the tester
- a misinterpretation of the result
- something wrong with the test environment
- something wrong with the baseline

Or it COULD BE a software fault.

Incident reporting

- The tester should stop and complete an incident report
  - describe exactly what is "wrong"
  - document the test script and script step identifiers
  - attach any outputs (screen dumps or printouts) that may be useful
  - impact on your current tests

Incident reporting

Bug tracking software:

- Reporting facility – complete with fields that will let you provide information about the bug, environment, module, severity, screenshots etc.
- Assigning – What good is a bug when all you can do is find it and keep it to yourself, right?
- Progressing through the life cycle stages – Workflow
- History/work log/comments
- Reports – graphs or charts
- Storage and retrieval – Every entity in a testing process needs to be uniquely identifiable, the same rule applies to bugs too. So, a bug tracking tool must provide a way to have an ID, which can be used to store, retrieve (search) and organize bug information.